

Randomness Compresses Local Timers: Exact State Complexity of Beacon-Triggered Broadcast in Anonymous Dynamic Networks

Ryutaro Yonezu
Independent Researcher

Preprint candidate — September 6, 2026

Abstract

We study a timer-based subclass of non-idle-start broadcast with stabilizing termination in anonymous, synchronous, 1-interval-connected dynamic networks. The subclass is motivated by the active-interval architecture used by Turau (SAND 2026): uninformed nodes repeatedly emit a constant-size beacon, quiet informed nodes start an active interval when they receive a beacon, and active nodes broadcast the information for a bounded interval.

Our first result allows the internal behavior of an active node to depend on received messages. For every deterministic beacon-triggered bounded-interval protocol on a known n -node network, the maximum possible active-interval length must be at least $n - 1$ rounds. Otherwise a moving-frontier adversary can keep one node uninformed forever by rotating its only informed neighbor through $n - 1$ informed nodes, always selecting a node whose previous active interval has expired.

We then specialize to locally timed active intervals, where the active-state transition is determined solely by persistent local state. In this natural local-timer subclass the interval lower bound becomes an exact state-complexity result: every correct deterministic protocol requires at least $n + 1$ persistent states per node, and $n + 1$ states suffice. Equivalently, the exact persistent memory requirement is

$$\lceil \log_2(n + 1) \rceil \text{ bits.}$$

The result does not lower-bound unrestricted deterministic non-idle-start broadcast. Rather, it isolates the precise cost of implementing the beacon-triggered active-interval paradigm with an exact deterministic local timer. Turau’s randomized Morris-counter construction realizes the same broad beacon/active-interval architecture with $O(\log \log n)$ memory with high probability, illustrating how randomization can compress this local timing mechanism.

1 Introduction

Broadcast in a dynamic network asks a distinguished broadcaster to disseminate a message to every node even though the communication topology may change adversarially from round to round. In anonymous networks without port labels, stabilizing termination is subtle: every node must eventually become informed and, after some finite time, all communication must cease.

Parzych and Daymude initiated a systematic study of local-memory requirements for terminating broadcast in anonymous, synchronous, 1-interval-connected dynamic networks [1, 2]. For stabilizing termination they proved that every deterministic *idle-start* algorithm needs $\omega(1)$ memory and gave an $O(\log n)$ -memory Countdown algorithm. They explicitly left the non-idle-start regime unresolved: when uninformed nodes may transmit from the initial round, can sublogarithmic memory be achieved, and can any lower bound be proved?

Turau answered the algorithmic side in the randomized setting [3]. His non-idle-start protocol makes each uninformed node broadcast an IDLE message in every round. A quiet informed node that receives IDLE begins an *active interval* during which it broadcasts INF. The interval length is controlled by a Morris-style approximate counter. This produces a stabilizing broadcast algorithm using $O(\log \log n)$ memory per node with high probability when n is known. Turau notes that simple deterministic solutions appear to require counting up to n .

This paper studies the deterministic cost of precisely that architectural idea. We deliberately do *not* attempt to solve the unrestricted deterministic open problem. Instead we ask:

What timer length is forced by the beacon-triggered active-interval architecture, and how many persistent states are required when that timer is implemented deterministically and locally?

Contributions. We separate the architectural and state-complexity parts of the question.

First, we define deterministic *beacon-triggered bounded-interval* (BTBI) protocols. An active interval may react arbitrarily to received messages, but an interval cannot be renewed while it is active and every active interval has a common finite upper bound L . We prove

$$\boxed{L \geq n - 1.}$$

The proof uses a moving-frontier adversary and does not rely on a message-oblivious timer.

Second, we define the locally timed subclass BTAI, in which the next active state depends only on the current active state. In this subclass, a terminating deterministic active trajectory cannot revisit an active state. Combining this finite-state fact with the interval lower bound yields

$$\boxed{S_{BTAI}^{\det}(n) = n + 1}$$

for every known $n \geq 2$. Hence the exact binary persistent-memory complexity is

$$\boxed{M_{BTAI}^{\det}(n) = \lceil \log_2(n + 1) \rceil.}$$

A matching protocol uses one uninformed state, one quiet state, and $n - 1$ active countdown states.

Finally, we place the result next to Turau’s randomized construction. The randomized counter may remain in the same internal counter state for many rounds and later escape using fresh randomness; this is exactly the behavior that a terminating deterministic local timer cannot emulate after revisiting a state. Thus the two results expose a concrete mechanism by which randomization compresses local timing state.

Scope. The exact theorem is a theorem about a natural algorithmic subclass, not about all deterministic non-idle-start protocols. In particular, it does not cover collective timers encoded across multiple nodes, protocols that renew activity while already active, protocols whose quiet nodes react to non-beacon traffic, or architectures using additional timing signals. The broader deterministic memory complexity therefore remains open.

2 Model

The node set V is fixed, $|V| = n$, and round t has an undirected snapshot $G_t = (V, E_t)$. An adversary chooses the snapshots subject to *1-interval connectivity*: every G_t is connected. Nodes

are anonymous, have no port labels, and execute the same deterministic transition system. In each round a node broadcasts the same message to all current neighbors.

For definiteness we use the following equivalent synchronous convention. At the start of round t each node has a persistent state. That state determines the message sent in round t . The node receives all messages sent by its current neighbors and then updates its persistent state for round $t + 1$.

There is one distinguished initial broadcaster. A node is *informed* if it is the broadcaster or has received the broadcast information INF from an informed node. Broadcast is complete once all nodes are informed. A protocol has *stabilizing termination* if every legal execution eventually has a suffix in which no node sends any message.

We work in the known- n setting: the transition system may depend on n . The *state complexity* is the number of persistent local states available to a node. A protocol with S states can be encoded using $\lceil \log_2 S \rceil$ persistent bits.

3 Beacon-Triggered Bounded Intervals

We first isolate the architecture needed for the moving-frontier argument. This class is intentionally broader than a message-oblivious local countdown.

Definition 1 (Beacon-triggered bounded-interval protocol). *For fixed n , a deterministic beacon-triggered bounded-interval (BTBI) protocol has a finite state set partitioned into semantic roles*

$$\Sigma = U \dot{\cup} Q \dot{\cup} A,$$

where U contains uninformed states, Q informed quiet states, and A informed active states. The protocol satisfies:

1. every state in U broadcasts IDLE;
2. every state in Q is silent;
3. every state in A broadcasts INF;
4. if a node in U receives INF, its next state is in A ;
5. if a node in Q receives at least one IDLE, its next state is in A , and if it receives no IDLE, it remains in Q ;
6. every maximal consecutive run of active states is an active interval; messages received while active may influence state changes, but cannot restart the interval without the node first visiting Q ;
7. there is a finite integer L such that every active interval in every legal execution has length at most L rounds.

The broadcaster starts active and every other node starts in U .

Condition 6 captures the non-renewal feature relevant to the active-interval architecture: an IDLE beacon may start a new interval when an informed node is quiet, but a currently active interval is not restarted on the fly. Condition 7 is uniform over received messages; the internal evolution of an interval may still depend on those messages.

Let $L(\mathcal{A})$ be the smallest valid bound L for protocol $\mathcal{A}$.

4 A Sharp Active-Interval Lower Bound

Lemma 1 (Moving frontier). *For every $n \geq 2$, any deterministic BTBI protocol that solves broadcast with stabilizing termination on all n -node 1-interval-connected dynamic networks satisfies*

$$L(\mathcal{A}) \geq n - 1.$$

Proof. For $n = 2$, the broadcaster must be active for at least its initial transmission, so the claim is immediate. Assume $n \geq 3$ and suppose for contradiction that $L \leq n - 2$.

Let the nodes be $v_1, \dots, v_n$, with v_1 the broadcaster. For rounds $0, 1, \dots, n - 3$, use the static path

$$v_1 - v_2 - \dots - v_n.$$

The broadcaster sends INF in round 0. Whenever v_i first receives INF, Definition 1(4) makes it active in the next round, so it sends INF then. Inductively, v_{i+1} becomes informed one round after v_i . Thus at the start of round

$$T = n - 2$$

the nodes $v_1, \dots, v_{n-1}$ are informed and v_n is still uninformed. We change the topology before transmissions in round T , so v_{n-1} does not send its first active-round INF to v_n .

For $j = 0, 1, 2, \dots$, define

$$q_j = v_{1+(j \bmod (n-1))}.$$

In round $T + j$, make the informed nodes $v_1, \dots, v_{n-1}$ induce a connected graph, for example a clique, and make v_n adjacent to exactly one informed node, namely q_j . Every snapshot is connected.

We show inductively that q_j is quiet at the start of round $T + j$. During the first cycle, $q_j = v_{j+1}$ begins its first active interval no later than round j . Because an active interval lasts at most $L \leq T$ rounds and cannot be renewed while active, it has ended by round $T + j$. After its right neighbor on the initial path became informed, that node no longer emitted IDLE, so no further interval was triggered before the frontier first reached q_j .

When a quiet q_j is used as the frontier in round $T + j$, it sends nothing, so v_n receives no INF. Node v_n does send IDLE, which may trigger a new active interval at q_j for round $T + j + 1$. The adversary immediately moves the frontier to q_{j+1} . The same node q_j is not used again for exactly $n - 1$ rounds. Its triggered interval has length at most $L \leq n - 2$, so it is quiet again by the time the frontier returns. This establishes the induction across all cycles.

Consequently, in every round from T onward, the unique informed neighbor of v_n is quiet during that round. Node v_n therefore remains uninformed forever even though every snapshot is connected. This contradicts correctness. Hence $L \geq n - 1$. $\square$

Remark 1. *Lemma 1 is an architectural lower bound, not yet a memory lower bound. Active-state transitions may depend arbitrarily on received messages. The lemma says only that a beacon-triggered non-renewing design must be capable of sustaining some active interval for at least $n - 1$ rounds.*

5 Locally Timed Active Intervals

We now impose the local-timer condition that converts interval duration into state complexity.

Definition 2 (Locally timed BTAI protocol). *A deterministic beacon-triggered active-interval (BTAI) protocol is a BTBI protocol for which the next state of an active node is determined solely by its current active state. Equivalently, there is a deterministic function*

$$f : A \rightarrow A \cup Q$$

that governs every active-state transition, independent of received messages.

This is the exact deterministic-local-timer abstraction: once an interval starts, the timer is represented entirely by the node's own persistent state.

Lemma 2 (No repeated active state). *If an active interval of a deterministic BTAI protocol has length ℓ , then it visits ℓ distinct active states. In particular,*

$$|A| \geq L(\mathcal{A}).$$

Proof. Suppose an active state $a \in A$ is visited twice during the same interval. Because the active transition is the deterministic function f and is independent of received messages, the trajectory from the second visit onward must repeat the trajectory from the first visit. The node is trapped in a cycle of active states and can never reach Q , contradicting the finite active interval required by Definition 1. Hence no active state repeats. $\square$

Theorem 1 (Exact deterministic state complexity). *For every known $n \geq 2$, the minimum number of persistent states required by a deterministic BTAI protocol that solves broadcast with stabilizing termination on all anonymous, synchronous, 1-interval-connected dynamic networks is*

$$S_{BTAI}^{\det}(n) = n + 1.$$

Consequently,

$$M_{BTAI}^{\det}(n) = \lceil \log_2(n + 1) \rceil \text{ bits.}$$

Proof. For the lower bound, Lemma 1 gives $L(\mathcal{A}) \geq n - 1$, and Lemma 2 gives $|A| \geq n - 1$. At least one uninformed state is required because every non-broadcaster starts uninformed, and at least one informed quiet state is required for stabilizing termination. The three roles are disjoint because they emit different outputs: IDLE, INF, and silence. Therefore

$$|\Sigma| = |U| + |Q| + |A| \geq 1 + 1 + (n - 1) = n + 1.$$

For the matching upper bound, use

$$\Sigma = \{U, Q, A_1, A_2, \dots, A_{n-1}\}.$$

State U broadcasts IDLE, state Q is silent, and every A_i broadcasts INF. The broadcaster starts in A_{n-1} and every other node starts in U . The transitions are

$$\begin{array}{ll} U \rightarrow A_{n-1} & \text{if INF is received, otherwise stay in } U, \\ Q \rightarrow A_{n-1} & \text{if IDLE is received, otherwise stay in } Q, \\ A_i \rightarrow A_{i-1} & \text{for } 2 \leq i \leq n - 1, \\ A_1 \rightarrow Q. & \end{array}$$

Active transitions ignore all received messages.

The broadcaster is active in rounds $0, \dots, n - 2$. Any node newly informed in one of those rounds enters A_{n-1} in the following round and remains active through the remainder of this initial $n - 1$ -round window. Hence, during each round $0, \dots, n - 2$, every informed node is active. If some node is still uninformed, 1-interval connectivity provides an edge crossing the informed/uninformed cut; its informed endpoint sends INF, so at least one new node becomes informed for the next round. Starting from one broadcaster, all n nodes are therefore informed by the start of round $n - 1$.

After that point no uninformed node remains to emit IDLE. Some quiet informed nodes may have been triggered by IDLE in the final progress round, but no new active interval can be triggered thereafter. Every current interval expires within $n - 1$ rounds and all nodes reach Q . Thus the protocol stabilizes and uses exactly $n + 1$ states. $\square$

6 Why Randomization Changes the Timer Cost

Turau’s 2026 randomized algorithm uses the same broad communication pattern that motivates our definitions: uninformed nodes broadcast IDLE, a quiet informed node that receives IDLE begins a new active interval, and it broadcasts INF during that interval [3]. The interval is controlled by a Morris-style approximate counter rather than an exact deterministic countdown. Turau proves $O(\log \log n)$ memory per node and high-probability stabilization for known n .

The state-repetition argument in Lemma 2 pinpoints why the two settings differ. In a deterministic local timer, revisiting the same active state forces the same future and therefore a permanent cycle. A randomized counter may revisit the same counter state for many rounds and later leave it using fresh randomness. Long random active intervals can consequently have unbounded support while the stored counter itself remains small.

Accordingly, the comparison is

$$\lceil \log_2(n+1) \rceil \quad \text{for deterministic exact local timing,}$$

versus Turau’s

$$O(\log \log n) \quad \text{for randomized approximate local timing with high probability.}$$

This is an architectural separation, not a lower bound for every deterministic non-idle-start algorithm.

7 Related Work

Parzych and Daymude established the first memory lower bounds for anonymous dynamic broadcast [1, 2]. For termination detection, any non-idle-start solution requires $\Omega(\log n)$ memory; for stabilizing termination, their lower bound is $\omega(1)$ for the idle-start regime. Their Countdown protocol achieves stabilizing termination with $O(\log n)$ memory and $O(n)$ time from idle start. Their conclusion explicitly identifies the non-idle-start stabilizing regime as requiring a different lower-bound approach.

Turau gives a randomized non-idle-start stabilizing-broadcast algorithm with $O(\log \log n)$ memory and constant-size messages for known n [3]. His Section 4 describes the IDLE/INF active-interval mechanism directly, and the paper remarks that the simplest deterministic solution broadcasts for n rounds after first receiving the information. Our contribution is not a new randomized algorithm; it is an exact state-complexity characterization for the deterministic locally timed version of this mechanism.

Approximate counting with tiny memory originates with Morris [4]; Nelson and Yu provide modern optimal bounds and a parameterized variant used in Turau’s construction [5].

A targeted literature search through September 6, 2026 did not identify a prior exact $n+1$ -state characterization for this beacon-triggered local-timer subclass. We therefore state novelty only for the explicitly defined subclass and do not claim resolution of the unrestricted deterministic problem.

8 Limitations and Open Problems

The local-timer restriction in Theorem 1 is essential. A general deterministic protocol may try to store timing information collectively in the configuration of several informed nodes, may renew or synchronize activity via additional messages, or may allow active-state transitions to use received traffic as an external clock. Lemma 1 still applies to the broader bounded non-renewing architecture, but Lemma 2 need not.

This leaves two concrete directions.

1. **Broader deterministic lower bounds.** Can one prove a superconstant, or logarithmic, memory lower bound for unrestricted deterministic non-idle-start stabilizing broadcast?
2. **Renewal classes.** How far can the exact state lower bound be extended if active transitions are allowed to react to non-beacon traffic while activity is still guaranteed to decay without a fresh beacon?

9 Conclusion

We isolated the deterministic cost of a natural beacon-triggered active-interval architecture for anonymous dynamic broadcast. Before any state-count argument is used, a moving-frontier adversary forces every correct bounded non-renewing design to support an active interval of length at least $n - 1$. When that interval is implemented as a deterministic local timer, its $n - 1$ active rounds require $n - 1$ distinct active states. Together with an uninformed state and a quiet state, this yields the exact bound $n + 1$.

The resulting memory requirement, $\lceil \log_2(n + 1) \rceil$ bits, contrasts with the doubly logarithmic memory achieved by randomized approximate counting in Turau’s active-interval construction. The result does not settle the broader non-idle-start problem, but it identifies a precise local timing barrier and a concrete mechanism by which randomness avoids it.

References

- [1] Garrett Parzych and Joshua J. Daymude. Memory Lower Bounds and Impossibility Results for Anonymous Dynamic Broadcast. In *38th International Symposium on Distributed Computing (DISC 2024)*, LIPIcs 319, 35:1–35:18, 2024. doi:10.4230/LIPIcs.DISC.2024.35.
- [2] Garrett Parzych and Joshua J. Daymude. Memory Lower Bounds and Impossibility Results for Anonymous Dynamic Broadcast. *Distributed Computing*, 39(3), Article 18, 2026. doi:10.1007/s00446-026-00511-4.
- [3] Volker Turau. Broadcasts in Anonymous, Dynamic Networks: A New Algorithm and Impossibility Results. In *5th Symposium on Algorithmic Foundations of Dynamic Networks (SAND 2026)*, LIPIcs 373, 6:1–6:17, 2026. doi:10.4230/LIPIcs.SAND.2026.6.
- [4] Robert Morris. Counting Large Numbers of Events in Small Registers. *Communications of the ACM*, 21(10):840–842, 1978. doi:10.1145/359619.359627.
- [5] Jelani Nelson and Huacheng Yu. Optimal Bounds for Approximate Counting. In *Proceedings of the 41st ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*, pages 119–127, 2022. doi:10.1145/3517804.3526225.